



DBMS HW 2

謝侑哲

112550069

1. Question Answering

A. Explain your method for inserting a key in the hash table. How do you solve the overflow problem?

```
void hash_table::insert(int key, int value){
    int idx = key & ((1 << this->global_depth) - 1); // depth 3 -> 111
    hash_bucket *bucket = this->bucket_table[idx];

    hash_entry* add = new hash_entry(key, value);
    if(bucket->first == nullptr){
        bucket->first = add;
        bucket->num_entries++;
    }
    else{
        bool same_key = false;
        hash_entry* now = bucket->first;
        if(now->key == key){
            now->value = value;
            same_key = true;
        }
        else{
            while(now->next != nullptr) {
                if(now->next->key == key){
                    now->next->value = value;
                    same_key = true;
                    break;
                }
                now = now->next;
            }
        }

        if(!same_key){
            now->next = add;
            add->pre = now;
            bucket->num_entries++;
        }
    }

    if(this->bucket_table[idx]->num_entries > this->bucket_size) this->extend(bucket_table[idx]);
}
```

對於 insert，我的方法如上，首先把 key 套上跟 global depth 等長的都是 1 的 mask，取出他結尾的二進制數值，命名為 idx。接著找到對應這個 idx 的 bucket，把 key 跟 value 變成一個 new entry 插入。插入方法首先檢查這個 bucket 有沒有值，沒有就直接插第一個，如果有的話，就一路遍歷到最尾端再插入。另外，如果有同樣的 key 值存在則直接代替掉原本的數值(保留最後出現的)，並且直接結束操作。

在 insert 完之後，我會檢查 bucket 裡面的 entries 數量，確認是否發生 overflow，如果發生 overflow 則對 bucket 使用 extend 如下：

```

void hash_table::extend(hash_bucket *bucket){

    if(bucket->local_depth == this->global_depth){
        this->bucket_table.resize(table_size << 1);
        for(int i=table_size; i < (table_size << 1); i++){
            this->bucket_table[i] = this->bucket_table[i-table_size];
        }
        table_size *= 2;
        this->global_depth++;
        this->num_local_glo_dpeth = 0;
    }
    int new_bucket_idx = bucket->hash_key | (1 << bucket->local_depth);

    hash_bucket* new_bucket = new hash_bucket(new_bucket_idx, bucket->local_depth+1);
    this->bucket_table[new_bucket_idx] = new_bucket;
    // !!!
    const int step = 1 << new_bucket->local_depth;
    for(int i = new_bucket_idx; i < this->table_size; i += step){ // 1XXX, 10XXX, 11XXX, 100XXX
        this->bucket_table[i] = new_bucket; // repoint to nearest
    }

    bucket->local_depth++;
    if(bucket->local_depth == this->global_depth) this->num_local_glo_dpeth += 2;

    hash_entry *now = bucket->first;
    hash_entry *next = nullptr;
    bucket->first = nullptr;
    bucket->num_entries = 0;

    while(now != nullptr){
        next = now->next;
        now->next = nullptr;
        now->pre = nullptr;
    }
}

```

```

if( (now->key & ((1 << bucket->local_depth)-1) ) == new_bucket->hash_key){ // 1XXX
    hash_entry* add = now;
    if(new_bucket->first == nullptr){
        new_bucket->first = add;
        new_bucket->num_entries++;
    }
    else{
        hash_entry* now = new_bucket->first;
        while(now->next != nullptr) now = now->next;
        now->next = add;
        add->pre = now;
        new_bucket->num_entries++;
    }
}
else{ // 0XXX
    hash_entry* add = now;
    if(bucket->first == nullptr){
        bucket->first = add;
        bucket->num_entries++;
    }
    else{
        hash_entry* now = bucket->first;
        while(now->next != nullptr) now = now->next;
        now->next = add;
        add->pre = now;
        bucket->num_entries++;
    }
}
now = next;
}
if(bucket->num_entries > this->bucket_size) this->extend(bucket);
if(new_bucket->num_entries > this->bucket_size) this->extend(new_bucket);
return;
}

```

首先檢查要 extend 的 bucket 的 local depth 是否跟 global depth 一樣，因為 local depth 在 extend 之後會+1，所以勢必在一樣的情況下要先拓展 global table，拓展方法就是直接 resize vector 並指向前面的 bucket (因為剛好差一個最左端的 1，也就是他一個 table size 的大小，所以直接減去 table size)，另外也更改 num_local_glo 這個用來記錄有多少 bucket 深度跟 global 一樣的為 0 (擴展後不會有一樣的 bucket)。

接著在原本 idx 的最左端加上一個 1，成為新分出去的分支 bucket，並且把兩個 bucket 的深度都+1 及維護 num_local_glo，接著把 table size 內原本指向原 bucket 的其他 idx 重新做指向，把應該屬於 new bucket 的那些重新指到 new bucket，判斷方法就是直接把 new bucket 的 hash key 直接在左端一直加數字，就會自然找到應該指向的。

最後把原本 bucket 裡面的元素重新分配，用跟 insert 一樣的方法重新判斷 key 並 insert 到對應的 bucket 中，如果分配完之後，有某個 bucket 還是 overflow 就遞迴下去繼續 extend。

B. Explain your method for removing keys from the hash table.

How do you shrink the bucket?

```
void hash_table::remove_query(vector<int> query_keys){
    for(int i=0;i<query_keys.size();i++){
        int key = query_keys[i];
        int idx = key & ((1 << this->global_depth) - 1);
        // bool flag = this->bucket_table[idx]->remove(key);

        hash_bucket *bucket = this->bucket_table[idx];
        hash_entry *now = bucket->first;
        hash_entry *next = nullptr;
        while(now != nullptr){
            next = now->next;
            if(now->key == key){
                if(now->pre != nullptr)
                    now->pre->next = now->next;
                else // only when delete first
                    bucket->first = now->next;

                if(now->next != nullptr)
                    now->next->pre = now->pre;
                delete now;
                bucket->num_entries--;
                break;
            }
            now = next;
        }

        if(this->bucket_table[idx]->num_entries==0 && this->bucket_table[idx]->local_depth > 1) this->shrink(this->bucket_table[idx]);
    }
}
```

對於 remove 元素的方法，首先對於要 remove 的 key 找到對應的 bucket，然後在這個 bucket 裡面遍歷，找看看有沒有一樣的 key，如果有就使用 link list 的刪除方法把前後的指向連起來，再把中間連接被砍斷的元素 delete 掉以確保記憶體不會洩漏，另外也有減少 num_entries，用來確認這個 bucket 是不是空的。

在刪除完之後，如果發現這個 bucket 是空的，那就啟動 shrink 如下：

```
void hash_table::shrink(hash_bucket *bucket){
    int idx = bucket->hash_key; // dep=3, 1XX or 0XX & 11 = XX
    int pair_bucket_idx = idx ^ (1 << (bucket->local_depth-1));
    hash_bucket *pair_bucket = this->bucket_table[pair_bucket_idx];

    if(pair_bucket->local_depth != bucket->local_depth) return;

    this->bucket_table[idx] = pair_bucket;
    if(pair_bucket->local_depth == this->global_depth) this->num_local_glo_dpeth -= 2;
    pair_bucket->local_depth--;
    pair_bucket->hash_key = min(idx, pair_bucket_idx); // leave the small one: 0XX < 1XX

    const int step = 1 << pair_bucket->local_depth;
    for (int i = idx; i < table_size; i += step) {
        this->bucket_table[i] = pair_bucket;
    }

    if(this->num_local_glo_dpeth == 0) {
        this->global_depth--;
        this->table_size /= 2;
        for(int i=0;i<this->table_size;i++){
            if(this->bucket_table[i]->local_depth == this->global_depth) this->num_local_glo_dpeth++;
        }
        delete bucket;
    }

    if(pair_bucket->num_entries==0 && this->bucket_table[idx]->local_depth > 1) this->shrink(pair_bucket);
    else if (pair_bucket->local_depth > 1 && bucket_table[pair_bucket->hash_key ^ (1 << (pair_bucket->local_depth - 1))]->num_entries == 0)
        this->shrink(bucket_table[pair_bucket->hash_key ^ (1 << (pair_bucket->local_depth - 1))]);
}
```

首先找到要 shrink 的 bucket 的對偶 bucket (pair bucket) · 接著確保兩者的 local depth 一樣 · 否則會發生 shrink 到更層的 bucket · 這會導致合併到深度不同的 bucket · 使得合併的過程中發生邏輯錯誤 · 接著把對偶 bucket 的東西直接放到原本的 bucket 裡面(原本的 bucket 是空的 · 而對偶是可能有東西的) · 接著把 hash key 的最左側那一位砍掉 · 這裡可以直接取最小值 · 因為最左位為 0 的一定比較小 · 之後再把 local depth 之類的 attribute 處理掉 · 並且用跟 extend 一樣的方式往後重新處理原本指向 bucket 的 · 讓他指向合併的 pair bucket 。

合併的過程中 · 如果 local depth 跟 global depth 一樣的話 · 那就把 num_local_glo 這個變數-2 · 這個變數是用來記錄目前有多少個 bucket 的 local depth 跟 global depth 一樣的 · 當他歸零時 · 就做全局的 shrink · 直接把 table 砍半 。

在東西都處理好之後 · 刪除空的 bucket 以確保記憶體不會洩漏 · 最後再判斷新合併的 bucket 會不會是空的(可能空的跟空的合併) · 或者是新合併的 bucket 的對偶 bucket 是不是空的 (當一空一滿 · 還是可以繼續往下做) · 如果發生的話 · 就往下地回去 shrink 。

C. There are other methods for managing key-value pairs, such as B+ trees. Please explain the basic idea of a B+ tree and describe the differences between B+ trees and extendible hashing. Which one do you think is better?

B+ Tree 的概念類似於 BST · 但是不是只有大於跟小於兩個分支 · 而是在一個 node 內存有很多的分界點(設 d 個) · 那這會產生 $d+1$ 個區間 · 對於每一個區間都可以繼續往下延伸出子節點 · 最終會把資料儲存在葉節點之中 。

他的優勢也就是非二元的特性 · 深度不會太深 · 還可以自動平衡 · 因此他再進行增刪查詢的時候 · 所需要的時間複雜度較低 · 可以用更快的速度做大量的操作 。

另外一個很大的優點是 · B+ Tree 可以做範圍查詢 · 因為他的資料在葉節點基本上就是依照 key 排序好的狀態 · 因此只要給定區間的頭跟尾並一路找到葉節點 · 這兩個葉節點之間的所有葉節點同時也是要找的其他點 。

B+ Tree 跟 Extendible Hash 主要的區別在於，B+ Tree 查詢需要 $O(\log_d(n))$ ，而 Extendible Hash 的查詢只需要 $O(1)$ ，並且 Extendible 因為沒有像 B+ 這麼多的引導點，每個點都是有儲存資料的，在記憶體の利用效率上更高。但 Extendible Hash 的資料幾乎是無序的，所以不能做到範圍查詢，需要一個一個去查。

所以要論哪個資料結構更好的話，端看使用情境，如果使用情境需要大量的範圍查詢，那 B+ Tree 的成本一定會更小，這時候就可以選用 B+ Tree，但如果不常使用範圍查詢的話，使用 Extendible Hash 的成本會更低。